Command Line

A modern introduction

Petr Stribny

Table of Contents

Command Line: A Modern Introduction

© 2021 Petr Stribny.

Version 2021-12.

Self-published.

https://stribny.gumroad.com/l/moderncommandline

Command Line

> I remember when the computer we had at home would boot into MS-DOS. To access Windows, one had to type “win” on the command line. The world has changed since then. Now we do the opposite.

Graphical user interfaces lowered the barrier of using computers for many people and enabled new applications to exist. However, in software development and system administration, *Command Line Interfaces* (CLIs) are still widely used. *Terminal User Interfaces* (TUIs) are used by interactive programs that go beyond just running commands while still operating from within a text-based terminal.

There are many benefits of using the command line:

- We can control systems that don’t have any graphical environment installed, including remote servers, virtual machines, and containers like Docker.
- Programs executed on the command line can be combined so that the output of one program can be the input of another.
- Task automation is built-in thanks to the scripting nature of many shells. No more doing repetitive tasks manually.
- There is a low barrier to creating our own CLI applications. Many utilities for programmers exist only as command-line programs.

Shells and Terminals

Shell is a term for graphical or text-based interfaces that we use to interact with operating systems. In other words, they are programs that provide a form of input and output to control computers. We often use a graphical user interface and thus a graphical shell. When we open a command-line window, also called *terminal*, a text-based shell is started for us in that window. Using this shell, we can control the computer by issuing text commands. Modern terminals work as *terminal emulators*, emulating text-based environments in a graphical interface.

> The name terminal comes from the era when computers were controlled by external devices called terminals. In the beginning, they were only able to accept text input from a keyboard and print results on paper. Later on, they also got computer screens and could be connected to remote computers and mainframes.

There are several text-based shells that we can use, but the most common and useful to know is *Bourne Again SHell*, known as Bash[1]. It is the shell commonly associated with Linux since it is installed by default on most Linux distributions, such as Ubuntu or Fedora. Another popular shell is Z shell[2] (Zsh), now the default shell on macOS. While Bash is an excellent choice for scripting thanks to its ubiquity, Zsh is used by developers and sysadmins for its interactivity and configurability. fish shell[3] is also gaining popularity, being designed to be a user-friendly shell with good defaults. However, it is less compatible with the other mentioned shells. Windows operating system also has its shell called *PowerShell*.

Depending on the operating system, we can choose between a number of terminal applications. In theory, we can combine different shells with different terminals and operating systems, but not all programs work with each other in practice. The following table presents some typical situations.

Table 1. Combination of operating systems, terminals, and shells

Operating System	Shell	Alternatives	Terminal	Alternatives
Fedora, Ubuntu, … (with Gnome)	Bash	Z shell and other Unix/Linux shells	Gnome's Terminal	Konsole, xterm, Tilix
macOS	Z shell	Bash and other Unix/Linux shells	Terminal	iTerm
Windows	PowerShell	console (legacy) or Bash, Z shell with Windows Subsystem for Linux	Windows Terminal	ConEmu, Console2, PuTTY

This book focuses on working with Bash and Z shells because of their practicality and popularity among programmers. All examples and programs are relevant to these shells unless explicitly said otherwise. Still, the programs and techniques can also serve as inspiration when using other shells.

Besides terminals, we can take advantage of *terminal multiplexers* like tmux[4] and *tiling terminal emulators* like Tilix[5] to open multiple shells in one window or screen. Terminal multiplexers can also detach and reattach a shell session which can be helpful in controlling remote processes.

Innovative applications are being created to bring new ideas to more traditional terminals and shells. Hyper[6] is a terminal application built entirely on web technologies. In contrast, XONSH[7] is a complete Python shell that understands Bash commands and allows the users to mix them both.

[1] https://www.gnu.org/software/bash/

[2] http://zsh.sourceforge.net/

[3] https://fishshell.com/

[4] https://github.com/tmux/tmux/wiki

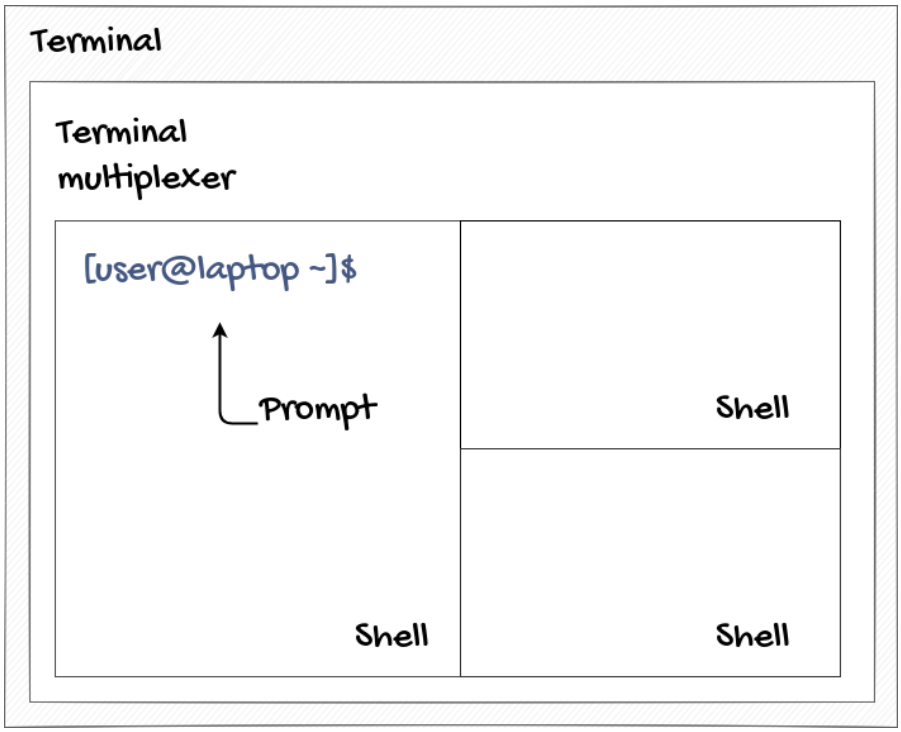
[5] https://gnunn1.github.io/tilix-web/

[6] https://hyper.is/

[7] https://xon.sh/

Getting Around the Command Line

Prompt



Opening a terminal will start a new shell session. We are then given access to a command line to execute commands. The beginning of the command line before the cursor is known as the *command prompt* (the `[user@laptop ~]$` in the picture). Command prompts are configurable and usually slightly different on different systems. Typically, the following is displayed:

- The user name of the user running the shell. The user affects all executed commands, as they will be run within this user's permissions.
- The name or an IP address of the computer where commands will be executed
- The currently selected directory (the `~` in the picture after the computer name). We will learn soon that a tilde refers to the user's home directory.
- The account type. By convention, the dollar sign (`$`) denotes a regular user, whereas

the hash (#) is used for system administrators. In documentation and various tutorials, these symbols might be used to distinguish whether something should be run under administrator privileges.

In this book, a prompt is marked with the greater than symbol (>) in all cases. If you are going to copy-paste something from the interactive examples, don't copy this symbol. Lines that don't begin with this symbol are outputs.

Running Commands

Running commands is as simple as typing them and hitting `Enter`. However, it is important to know where a command is executed in the file system. Typically, a new shell session will start at our home directory like `/home/username` where *username* will be the name of our user on the system. Many shells display the current file system location inside the prompt. Either way, we can simply see where we are by running the command `pwd` (print working directory):

```
> pwd
/home/username
```

A command like this seems straightforward, but it is not. The problem is that a command can run a script, an executable binary, a custom shell function, an alias, or a *shell built-in* (built-ins are utilities implemented directly in the shell itself). We will see all of those things in action later. For now, let's find out what has been run.

The easiest way to do that is with the command `type -a pwd`. It will print all executable things named `pwd`.

```
> type -a pwd
pwd is a shell builtin ①
pwd is /usr/bin/pwd ②
```

① `pwd` is a shell built-in. Because it is a shell built-in, it will take precedence over external programs. Notice that the output mentions it first.

② `pwd` is also a separate executable program in `/usr/bin/pwd`.

It turns out that `pwd` is two separate things on the system. Even though they both serve the same purpose, they are, in fact, different. If the built-in command weren't part of the shell, the program at `/usr/bin/pwd` would get executed instead. But how does the shell know where to look for programs? Can we expect the shell to find all programs on the file system?

Shells will search for executable programs in a list of locations called the *PATH*. On Linux, the PATH looks something like this:

```
/home/username/.local/bin:/usr/share/Modules/bin:/usr/local/bin:/usr/local/sb
in:/usr/bin:/usr/sbin:/home/username/bin:/var/lib/snapd/snap/bin
```

We can see that the PATH is just a list of file system paths separated by semicolons. Any program in one of these locations can be executed without specifying its location. Because `/usr/bin` is mentioned here, the shell can find `/usr/bin/pwd` just based on the file name. If multiple programs have the same name, the order of the paths will be taken into account.

What can we do if a program is not on the PATH, or when we need to run a program overshadowed by another program or built-in of the same name? We need to run it by typing the absolute or relative file path.

```
> /usr/bin/pwd ①
/home/username ②
```

① We run the exact program by specifying the full path to the executable.

② In this case, the output of the program is the same as it was from the built-in.

The PATH is stored as a *process environment variable*, exposed in a shell as a *shell variable*.

Variables and Environment Variables

A shell variable has a case-sensitive name made out of a sequence of alphanumeric characters and underscores. It stores a value, usually a string or a number, and can have several attributes. For instance, a variable can be marked as read-only or having a

particular type, although working with these attributes is often unnecessary.

Variables can be defined or reassigned using the equal sign (`=`) and used with the help of *variable substitution*. When a variable is prefixed by the dollar sign (`$`) in a command or inside a double-quoted string, it will be replaced by its value. To see it in action, let's introduce another built-in shell command, `echo`. It simply prints anything passed to it.

```
> my_var=10 ①
> echo $my_var ②
10
> echo "My variable value is $my_var" ③
My variable value is 10
```

① Variable assignment. Notice that there is no space around the equal sign (`=`).

② The shell will substitute `$my_var` with the value of `my_var` and pass this value to `echo`. `echo` prints it.

③ Variable substitution used inside a double-quoted string.

The full syntax for variable substitution uses curly braces. So the example would look like `echo ${my_var}`. This full syntax is useful to know if we ever need to place other characters directly around the variable.

Environment Variables

Process environment variables (also *env vars*) are not the same as shell variables. They are configuration variables that every process has in Unix-like operating systems. As a shell session is also a process, it receives some env vars from the system when it is run. The shell imports them at startup time as regular variables but marks them as *exported*.

Every variable marked as exported will be copied into the process environment of all child processes started in the shell. This behavior makes it possible to pass configuration and secrets to all programs we run.

To see all variables marked as exported (which will eventually become environment variables of the child processes), use the command `export`.

```
> export
PATH=/home/username/.local/bin:/usr/share/Modules/bin:/usr/local/bin:/us
r/local/sbin:/usr/bin:/usr/sbin:/home/username/bin:/var/lib/snapd/snap/b
in ①
...
```

① All exported variables are printed. The shell set them as environment variables automatically, or we exported them ourselves before. The output is shortened, only showing that the PATH variable is included.

Useful environment variables set by the system include `PATH`, `USERNAME`, and `HOME`. Since the shell imported them all as variables when it started, we can print their values the same way we did before.

```
> echo $PATH
/home/username/.local/bin:/usr/share/Modules/bin:/usr/local/bin:/usr/local/sb
in:/usr/bin:/usr/sbin:/home/username/bin:/var/lib/snapd/snap/bin
> echo $USERNAME
username
```

To promote a regular variable into an environment variable, we need to export it. A variable can be exported and assigned at the same time:

```
> ENV_VAR='Just a regular variable now' ①
> export ENV_VAR ②
> export ENV_VAR_2='Already an environment variable' ③
```

① Standard variable assignment as before. Using single quotes to denote an exact string is safer when variable substitution is not needed.

② Export of an existing variable. `ENV_VAR` will now be passed as an environment variable to programs started in the same shell session.

③ Export of a completely new variable.

Now we can change the PATH. It will make it possible to execute programs on a command line just by their name from any location. For instance, we can create a new `bin` folder inside a home directory to store programs and scripts and include it on the PATH for convenience:

```
> export PATH="$HOME/bin:$PATH" ①
```

① We set `PATH` to a new value, using variable substitution to use both the path to our home directory and the previous value of `PATH`. We export it to make it available to child processes as well.

After changing the `PATH`, we can place any executables inside the `/home/username/bin` directory and execute them by typing just their name. However, this change is not permanent and will be gone when the current shell session is closed. The *Configuration* section later in the book demonstrates how to make it permanent.

Absolute and Relative Paths

File paths that start with a forward slash `/` are absolute, fully determining the location of a file or a folder. We can also use relative paths relative to the location of the working directory. Such paths start with one or two dots, pointing to the current or parent directory. The *tilde expansion* provides quick access to the home directories when the path starts with `~`. See the table below:

Table 2. Referring to files and folders

The path starts with	Points to

`/`	The root directory of the file system
`.`	The current directory
`..`	The parent directory
`~`	The home directory of the current user
`~username`	The home directory of the user `username`

Changing the current working directory can be done with the built-in command `cd` (change directory). Now consider a program named “myprogram” in the home folder `/home/username` and assume that it will output a simple hello message. We can execute it from various locations like this:

```
> # Assuming the folders /home/username/folder/folder2 exists ①
> cd ~/username ②
> pwd ③
/home/username
> ./myprogram ④
Hello!
> cd /home
> ./username/myprogram
Hello!
> cd /home/username/folder
> ../myprogram
Hello!
> cd /home/username/folder/folder2
> ../../myprogram ⑤
Hello!
```

① Every line that starts with `#` is treated as a comment in Bash and Zsh. In other words, it doesn’t do anything.

② We use `cd` with a tilde expansion to move to the `username` home directory.

③ We can check that the working directory was changed with `pwd`.

④ It is a good practice to run programs in the current directory with `./` prefix to avoid name collision with other programs on the PATH.

⑤ We can traverse to parent directories by repeating `../`.

The shell also stores the current and previous working directories in variables called `PWD` and `OLDPWD`. Running `cd -` can switch to the previous working folder directly. Running it multiple times will keep switching between the last two working directories.

Arguments

We have already been using arguments when running previous commands. Technically speaking, all tokens separated by a space on the command line are *command line arguments*, the first being a utility name or a script we want to run. Once a shell parses a command, it will know what to execute based on this first argument. The program, function, or built-in being called will use the rest of the arguments as parameters for its execution.

Because spaces separate arguments, all spaces within an argument need to be escaped by a backslash. Alternatively, we can quote them with single or double quotes, as we saw already with strings. Single quotes are used for exact strings, while double quotes allow for various substitutions.

Arguments that are passed one after another are called *positional arguments*. Programs and scripts can access such arguments based on the numerical position or iterate over all of them.

```
> echo Hello World ①
Hello World ②
> cd directory\ with\ spaces ③
> cd 'directory with spaces' ④
```

① The first argument is `echo`, determining what will be called. The following "Hello World" is, in fact, two arguments separated by space. This means that the built-in command `echo` is passed two positional arguments, "Hello" and "World."

② `echo` prints all its positional arguments one after another.

③ The first argument is `cd`. The shell will call this built-in with one positional argument because the spaces are escaped. Assuming the directory exists, the working directory will be changed to "directory with spaces."

④ It is usually preferable to quote such arguments for readability.

Besides positional arguments, there are also two types of named arguments. They are

referred to as *options*, meaning that they are usually optional and change how the program behaves. Short options start with a hyphen (`-`), followed by a single character. The long ones begin with two hyphens (`--`), followed by the option name.

For demonstration, let's look at a program called `ls` (list directory). `ls` prints the contents of directories that are passed as positional arguments. Without any arguments, it prints the contents of the current working directory. With `-a` and `-l` options, we can change the output:

```
> ls ①
dir1 file1.txt file2.pdf
> ls -a ②
. .. .gitignore dir1 file1.txt file2.pdf ③
> ls --all ④
. .. .gitignore dir1 file1.txt file2.pdf
> ls -l ⑤
total 12
drwxr-xr-x.  3 groupname username 4096 Dec 12 15:05 .
drwxrwxr-x. 15 groupname username 4096 Dec 10 16:56 ..
drwxr-xr-x.  2 groupname username 4096 Dec 12 15:05 dir1
-rw-r--r--.  1 groupname username    0 Dec 12 15:05 file1.txt
-rw-r--r--.  1 groupname username    0 Dec 12 15:05 file2.pdf
```

① `ls` prints what is inside the current working directory. By default, hidden files are not shown (on Unix and Linux-based systems, that means all files beginning with a dot `.` are omitted).

② A short option `-a` will tell `ls` to display all files.

③ Besides the `.gitignore` file, we can also see the references to the current and parent directories (`.` and `..`). It is because the references are implemented using *symlinks*, which are also files.

④ We use a long-form option called `--all` to tell `ls` to display all files. It is just a variation of the short option `-a`. It is often the case that many named arguments have both a short and a long variant.

⑤ The output when using `-l` option displays additional information about the files and directories like the type, permissions, size, modified date, and so on. We will look at this output again in more detail in the section *Accounts and Permissions*.

Positional and named arguments can be combined in one command (e.g., `ls -l --all`).

Short arguments can sometimes be written together (`ls -la` vs. `ls -l -a`). Options are typically written before positional arguments (`ls -l ~/`).

Many applications store their configuration in hidden files that start with a dot. They are called *dotfiles*. Dotfiles can be typically found in the user home directory `/home/username` for user-specific configuration or directly in the application folder. `.gitignore` file for version control system Git is an example of a dotfile.

Options can be used as flags or take an *option argument* (again, separated by a space). Long options can also take a value as `--argument-name=value`. Although programs that follow the POSIX[8] standard share some common characteristics, there is no standard in handling program arguments. So it is always best to consult the documentation of each program.

Command History and Autocompletion

While Bash stores the command history in a file by default, in Zsh, it has to be enabled by setting the appropriate variable (`HISTFILE=~/.zsh_history`).

Both Bash and Z shells can preserve command history (all commands that we issued on the command line). By pressing `Up` and `Down` keys, we can rotate between previously entered commands. It is especially useful if we want to run a command that we had run recently.

The last executed command can be reused with two exclamation marks (`!!`) that will automatically expand it. Another option is to prepend a name of the command with one exclamation mark (`!`), which will execute the same program with the same arguments as the last time. For instance, to execute `ls -al` again, use `!ls`.

There is also a trick to insert the last argument of previous commands. This trick is useful since many programs have a file path as their last argument. Pressing `Esc` + `.` or `Alt` + `.` will rotate the last arguments of previous commands:

```
> ls path/to/directory ①
> cd ②
> cd path/to/directory ③
> cd !$ ④
```

① `ls` utility is used with a folder path as an argument.

② After seeing the contents of the directory, we decide this is the right directory to move to and type `cd` followed by a space.

③ Pressing `Alt` + `.` will autocomplete the path.

④ `cd !$` will do the same.

Pressing `Ctrl` + `R` will open *interactive search* of the command history. Once open, type a portion of the command to be retrieved. Pressing `Ctrl` + `R` repeatedly will navigate between matches.

McFly[9] is an "intelligent" utility that takes over the interactive history search. It shows the most important history entries based on our directory and the execution context of what we typed before. In other words, it tries to guess the command we are interested in.

History Command

A command named `history` lists things we have executed in the past.

```
> history
1968  ls
1969  ls -al
```

We can then execute any previous command again using an exclamation mark and the printed number. So to run `ls -al` again, all we have to do is to type `!1969`, followed by `Tab` or `Enter`.

Commands inside comments (starting with `#`) are stored in the history as well. This behavior can be leveraged to store complete or incomplete commands for later use.

The complete history is stored in the history files `~/.bash_history` for Bash and `~/.zsh_history` for Z shell or other location of our choosing. This behavior might be problematic when issuing sensitive commands, but fortunately, there are ways to avoid storing current commands in history. In Bash, putting one space character before the command is enough, and the same behavior can be enabled in Zsh with `setopt HIST_IGNORE_SPACE`.

Both shells also offer command auto-completion by pressing `Tab` when typing commands. Both can suggest command names and files where files are expected or auto-complete argument names. Z shell has an advantage since it offers a nice little menu to cycle through possible options interactively. In Bash, bash-completion[10] project can be installed to enhance the default behavior.

To automatically correct misspelled commands, we can use a program called thefuck[11]. It has various rules for some commonly used programs and usages. By running `fuck` on the command line, it will correct the last entered command.

Using thefuck to correct misspelled commands

```
> cdd ~/ ①
bash: lss: command not found...
> fuck
cd ~/ [enter/up/down/ctrl+c] ②
```

① The intention is to run `cd ~/`, but it was misspelled to `cdd`.

② After running `fuck`, we are offered the correct command and can confirm it by pressing `Enter`.

Programs mentioned with a link will need to be installed first. The book won't distinguish between built-in commands, standard utilities, and external programs from now on.

Shell Expansions

Besides the tilde expansion that we have already seen, there are some other interesting expansions available that can save us a lot of time.

When referring to files and folders, we can use the *file name expansion* which replaces a wildcard pattern with all matching file paths (also called *globbing*). The basic wildcards are `?` for exactly one character and `*` for zero or multiple characters. Let's imagine that there are two text files, one containing `Contents of A` and the other containing `Contents of B`. We can then print their contents with a program called `cat` that accepts multiple files as its arguments or use the file name expansion to do the same.

```
> cat a.txt b.txt ①
Contents of A
Contents of B
> cat *.txt ②
Contents of A
Contents of B
```

① Similar to `echo`, `cat` performs its operation with all positional arguments. In this case, the contents of both files will be printed to the console.

② Using a globbing pattern instead, it is enough to provide just one argument containing the pattern.

Besides wildcard characters, file globbing supports multiple different matching mechanisms, similar to regular expressions:

- Matching specific characters from options written inside brackets, e.g., `[ab]` (a or b)
- Matching characters from a sequence written inside brackets, e.g., `[0-9]` to match a number character or `[a-d]` to match a, b, c, or d
- Caret (`^`) character to negate the selection
- Dollar sign (`$`) to denote the end of the name
- Choosing between several patterns using `(PATTERN|PATTERN2)`

All of them can be combined to arrive at more complex patterns. See the table below for examples.

Table 3. Globbing examples

Pattern	Explanation
`backup-*.gzip`	Match all files starting with `backup-` that have `.gzip` file extension.
`[ab]*`	Match all files that start with either `a` or `b`.
`[^ab]*`	Match all files that don't start with `a` or `b`.
`*.???`	Match all files whose file extension has exactly three characters.
`(*.???|*.??)`	Match all files whose file extension has exactly two or three characters.

The asterisk character (`*`) will by default not match any hidden files starting with a dot. To prevent Bash or other shells from performing globbing, it is needed to either escape the characters with a backslash (`\*`) or quote them.

The *brace expansion* allows composing names from multiple variants by placing them in curly braces like `{variant,variant2}`.

Creating multiple files with different names, but the same extension

```
> touch {file1,file2,file3}.txt ①
> ls
file1.txt file2.txt file3.txt ②
```

① The original purpose of the program `touch` was to update modified timestamps on files, but it can be also used to create empty files.

② Three files were created, sharing the same file extension.

Aliases

We can create an alias when we don't want to type or remember complex commands. An alias can point to anything that would be typed on a command line.

```
> alias project1="cd /home/username/project1" ①
> project1 ②
> pwd
/home/username/project1 ③
```

① An alias called `project1` is set to change the working directory to a project folder.

② The alias can be invoked as any other program just by typing its name.

③ The command was executed through the alias.

An alias will take precedence over external programs and even built-in commands. That means we can also overshadow standard utilities like `cd` or `ls` and replace them with something else.

Standard Input, Standard Output, Standard Error

Standard input, output, and error streams, also referred to as `stdin`, `stdout`, and `stderr`, are created when a command is run. These are text data streams that applications can use to pass data to each other. Many standard utilities and command-line applications work with input or output streams. The error stream is a separate stream where a program can output error messages that can also be redirected to another program or a file. The streams behave like files and are identified by file descriptors, 0 being the input stream, 1 being the output stream, and 2 being the error stream. The streams don't have

any specific format and can contain any text.

We have already seen the standard output stream in action. When commands such as `echo` or `cat` are executed, their standard output is displayed in the shell. It is the default behavior for standard output when a program is run from a command line. We can redirect this output stream using `>` and `>>` operators. A classic example is saving the output of the stream to a file:

```
> echo "Hello!" > file1.txt ①
> cat file1.txt
Hello! ②
> echo "Hello!" >> file1.txt ③
> cat file1.txt
Hello!
Hello!
```

① We will not see any output from the `echo` command in the terminal, because the output got redirected to a file.

② We can check that there is now a file `file1.txt` with the contents "Hello!".

③ With `>>` operator, we can append text to a file instead of overwriting it.

Another Example of Standard Output Redirection

```
> cat header.csv spreadsheet.csv > combined.csv ①
```

① Multiple files can be merged into one. `cat` will read and output all files passed to it, while the operator will redirect the whole output to a new file.

We can redirect the standard error stream similarly, identifying the stream by its file descriptor.

```
> cat file_which_does_not_exist.txt
cat: file_which_does_not_exist.txt: No such file or directory ①
> cat file_which_does_not_exist.txt 2> error.txt ②
> cat error.txt
cat: file_which_does_not_exist.txt: No such file or directory
```

① The standard error stream is also visible in the shell by default.

② We can prepend the `>` or `>>` operators with the descriptor identifying the stream, in this case, 2 for `stderr`, to redirect the stream to a file. Again, no output will be visible in the shell now.

The error stream will be printed to the terminal when we redirect just the standard output. If we redirect just the error stream, the standard output will be printed in the terminal. Note that a program can use both streams at once.

It is also possible to redirect both the standard output and the error stream at once by chaining the operations together or by using the ampersand (`&`) instead of the stream number. Let's assume that we want to run a script called `streams.sh` that writes into both streams:

```
> ./streams.sh 1> output.txt 2> error.txt ①
> ./streams.sh &> output_and_error.txt ②
> ./streams.sh &> /dev/null ③
```

① Both standard streams are captured in separate files by placing the redirects one after another.

② Alternatively, we can capture the streams into the same file. `&>` will overwrite the file while `&>>` will append to it.

③ We can discard all output of a program by sending the outputs to `/dev/null`, which is something like a black hole. In this case, the script or program will run without its output being outputted to the screen nor saved in any file.

A pipe operator (`|`) is used to pass the standard output stream to another program as the input stream. Piping programs together is the most common way to compose a command that needs different programs and functions to work.

```
> echo "Hello, NAME" | sed "s/NAME/Petr/"
Hello, Petr
```

In this example, the standard output of `echo` is fed into `sed`, a utility for transforming text. Sed has a "replace" operation that expects a regular expression to match a text and replace it with something else, in our case matching `NAME` and replacing it with `Petr`. We will go into more detail later in the section *Text Data Wrangling*. For now, it is enough to understand how text data flows from one program to another.

To pipe all streams at once, including the `stderr`, we can replace `|` with `|&`. This change will put both streams on the standard input for the consumption of the following program.

The utility `tee` can split the output of a program. It redirects it to multiple files and/or to the screen. It is handy when we want to save the output to a file but see it in the terminal simultaneously. Building on our previous example, we will now use `echo` together with a pipe and `tee` to get the output both to the screen and to multiple files at once:

```
> echo "Hello!" | tee file1.txt file2.txt
Hello!
> cat file1.txt
Hello!
> cat file2.txt
Hello!
```

Tee can also append the output to a file instead of overwriting it with `-a` option. This is useful when we need to check a program's output multiple times and keep all the history, e.g., when using a profiler.

Finally, the opposite operators `<` and `<<` are used to redirect the standard input, in other words, to send some text data to a program. However, using these operators is not that common because it is better to use the command line arguments or pipes to achieve the same thing in most situations.

Running Multiple Commands

It is possible to run multiple commands consecutively and conditionally with `&&`, `||` and `;`

operators. A semicolon is just a *command delimiter*, allowing us to place multiple commands on the same line. `&&` and `||` are *short-circuiting operators* that we know from other languages. They will only evaluate the following command if the preceding one will exit with a true or a false value.

```
> true && echo "This will be executed"
This will be executed
> false || echo "This will be executed"
This will be executed
> false ; echo "This will be executed"
This will be executed
```

The evaluation of whether the run of a command is true or false is based on *exit codes*. We will see how it works in the section *Programming in Bash*.

Using xargs

A program called `xargs` can be used to execute a program multiple times. It reads the standard input and converts it into arguments for another program.

```
> ls
a.png  a.txt  b.png  b.txt
> find . -name "*.png"
./a.png
./b.png
> find . -name "*.png" | xargs rm -rf
> ls
a.txt  b.txt
```

`find` is a standard utility that can find files. The location where to look for them is passed as a positional argument, while the searching pattern is specified using the `-name` option. Consult the section *Search* for more information about finding files.

`rm -rf` is used to delete files on a disk recursively. The typical usage would be to call it with arguments (`rm -rf a.png b.png`), but in this example, it is invoked through `xargs`. `xargs` converts the input stream from `find` into arguments for `rm`.

Shell Shortcuts

Various shell shortcuts make it possible to operate quickly on a command line when entering commands. The table lists basic shortcuts that work in both Bash and Zsh.

Table 4. Common shell shortcuts

Operation	Shortcut
Rotate previously entered commands	`Up` and `Down`
Go to the beginning of the line	`Ctrl` + `A`
Go to the end of the line	`Ctrl` + `E`
Go back one word	`Alt` + `B`

Operation	Shortcut
Go forward one word	`Alt` + `F`
Delete the previous word	`Alt` + `Backspace`
Delete the next word	`Alt` + `D`
Cut from the cursor to the end of the line	`Ctrl` + `K`
Open history search	`Ctrl` + `R`
Clear the screen	`Ctrl` + `L` (or type `clear`)
Cancel the command execution with `SIGINT` signal	`Ctrl` + `C`
Cancel the command execution with `SIGQUIT` signal	`Ctrl` + `\`
Pause the command execution with `SIGSTOP` signal	`Ctrl` + `Z`
Close the shell session	`Ctrl` + `D` (or type `exit`)

The `SIGINT`, `SIGQUIT` and `SIGSTOP` signals will be explained later in the section *Job Control*.

Many of these shortcuts are concerned with text editing. This functionality is provided by the GNU Readline library, which offers two editing modes, `vi` and `emacs`, based on the editing style of the text editors of the same names. By default, `emacs` editing style is used. It means that the text editing shortcuts that we learn for the shell can be utilized directly in Emacs. On the other hand, if we already use and know vi (or its successor Vim), we can switch the editing mode of the shell to support vi shortcuts instead. This change is done by executing `set -o vi` (set it in the shell configuration file). More information about this can be found in the GNU Readline library documentation[12].

Terminal emulators or multiplexers will also have their own set of valuable shortcuts worth learning for tasks like switching between terminal sessions.

Getting Help

The standard way to ask programs for help is to use `-h` or `--help` options. Many programs will understand this convention and print their documentation. As an example, look at the output of `cat --help`.

The `man` program displays manual pages for many standard utilities like `cat` or `find`. Man uses the program called `less` designed for page-by-page browsing under the hood. To test it, we can display the manual page of the man command with `man man`, or provide another program name as the argument. The documentation will be opened in `less` by default. See the table for some tips on how to navigate the docs comfortably.

Table 5. Navigating in less, e.g., when browsing manual pages

Operation	Command/shortcut
Quit	`q`
Display help for navigating in less	`h`
Navigate up and down	`Up` and `Down`
Navigate pages up and down	`PageUp` and `PageDown`
Go to the beginning	`g`
Go to the end	`G`
Find a text located after the cursor's position	`/` + type pattern
Find a text located before the cursor's position	`?` + type pattern
When multiple patterns are found, move to the next one	`n`
When multiple patterns are found, move to the previous one	`N`

We can use `less` to browse any large text file and use the same interface and shortcuts as we do when viewing manual pages. Just execute `less` explicitly with the text file as the argument (`less file.txt`).

Explain Shell[13] is an online application that will explain any combination of standard commands that we come across and don't understand. The advantage is that we don't

have to look up documentation for individual commands separately. It is enough to copy and paste the entire command and Explain Shell will explain it.

Cheatsheets

"Cheatsheet" utilities provide quick help on the common usage patterns. TLDR.sh[14] is one of them. It can be installed as a command-line application or used online.

Using TLDR

```
> tldr echo
  echo

  Print given arguments.
  More information: https://www.gnu.org/software/coreutils/echo.

  - Print a text message. Note: quotes are optional:
    echo "Hello World"

...
```

navi[15] is a tool for creating custom cheatsheets and executing them interactively. It makes it easy to store help texts in files and access them on the command line. Community-sourced command patterns from the TLDR database can be accessed with `navi --tldr`.

[8] https://en.wikipedia.org/wiki/POSIX

[9] https://github.com/cantino/mcfly

[10] https://github.com/scop/bash-completion

[11] https://github.com/nvbn/thefuck

[12] https://tiswww.cwru.edu/php/chet/readline/rltop.html

[13] https://explainshell.com

[14] https://tldr.sh/

[15] https://github.com/denisidoro/navi

Configuration

Changing Shell

We can switch between several installed shells simply by typing their name. So when we are in Bash, we can type `zsh` to enter Z shell and type `bash` to enter Bash. This approach starts a new process every time, but it is the quickest way to do it if necessary.

To make a choice of shell permanent, we need to use a program called `chsh` (change shell).

Change default shell with chsh

```
> chsh -s $(which zsh) ①
```

① A *command substitution* is used to replace `which zsh` with its result. `which` finds the path to Zsh. If Zsh is installed, it will be set as the default shell.

All shells have *startup files* or *startup scripts* that are executed as soon as a new shell session starts. Because a shell can start in an interactive or a non-interactive mode (e.g., when running a script from a file), there are different startup files that we might want to use.

This section only covers the interactive mode, which is used when the shell runs from a terminal. Bash and Zsh shells store their startup files in the user's home directory. Bash in `~/.bashrc` and Zsh in `~/.zshrc`. The script itself is just a file with commands written on new lines, so we can reuse everything we have learned so far.

In startup scripts, we usually want to:

- Set shell settings (called *options*)
- Export environment variables
- Configure the prompt
- Change themes

- Set up custom aliases
- Define custom functions

The following example demonstrates how a startup script can look like. Please note that not all the settings are compatible with Bash and that this example expects Oh My Zsh to be installed.

An example ~/.zshrc config for Z shell

```
# SHELL OPTIONS
HISTSIZE=10000000 ①
SAVEHIST=10000000
setopt EXTENDED_HISTORY

# ENVIRONMENT VARIABLES
export EDITOR="vim" ②
export PATH="$HOME/bin:$PATH" ③

# PROMPT
export PS1="> " ④

# THEME
ZSH_THEME="af-magic" ⑤

# ALIASES
alias dirs="colorls -dla" ⑥
alias files="colorls -fla"
```

① We set how many commands are saved in the command history and how many of them should be loaded in memory. Zsh also offers an “extended” mode for storing command history that records the timestamp of each command.

② We set the default application for editing files by setting `EDITOR` env var. When a program like Git prompts us to write a message on the command line, the shell will open our favorite editor.

③ Adding new paths to `PATH` is probably the most common configuration in startup scripts. Some applications edit shell startup scripts as part of their installation to modify the PATH.

④ Configuring prompt is done by setting the `PS1` variable. This is not a good value. However, it would recreate the look of the prompt from this book.

⑤ We can change the theme of the shell when using Oh My Zsh.

⑥ We create new aliases to list files and folders that are easy to remember. In this case, the program `colorls` has to be installed.

When making changes to a startup script, it is necessary to re-apply the configuration using the command `source` (`source ~/.zshrc`). Otherwise, the changes would be visible only in a new shell session.

It is possible to configure what is shown in a command prompt by adjusting the environment variable `PS1` as demonstrated by Easy Bash Prompt Generator[16]. Powerline[17] is a helpful plugin for Bash, Z shell, Vim, and other programs that displays more information in a beautiful way.

Coming up completely with our configuration can be time-consuming. This is where frameworks like Oh My BASH[18] and Bash-it[19] for Bash and Oh My Zsh[20] for Z shell come handy. They include sensible default configurations and come with useful functions, plugins, and themes ready to be used.

Reusing Shell Dotfiles

It is good to keep the most important and customized dotfiles in one place and under version control. The idea is to create a "dotfiles" folder in a home directory, version it, and store it online. To do that, we need to:

- Link dotfiles to a new location using symlinks, use the `source` command, or `[include]` section to load configuration from an external location.
- Set our aliases and configuration conditionally if we want to reuse them in different shells or on different operating systems.
- Version external configuration using a version control system like Git.

The possibility to include one configuration file in another can be used for loading a file from a different location and dividing configuration into multiple smaller files. Let's consider that we want to load our custom `~/dotfiles/.aliases` file from `~/.bashrc` or `~/.zshrc`:

Place this inside a startup script to load external configuration

```
# using source with a check whether the file we want to source exists
if [ -f ~/dotfiles/.aliases ]; then
    source ~/dotfiles/.aliases
fi

# using [include]
[include]
   path = ~/dotfiles/.aliases
```

If we need to make our startup scripts more flexible, we can run code conditionally based on the type of shell, the operating system used, or even based on a specific machine:

```
# example: apply only on Linux
if [[ "$(uname)" == "Linux" ]]; then
    # place the code here
fi

# example: set a Zsh theme only in Z shell
if [[ "$SHELL" == "zsh" ]]; then
    ZSH_THEME="af-magic"
fi

# example: set a ~/bin folder on path only on a specific computer (in
this case, the computer's hostname is set to laptop)
if [[ "$(hostname)" == "laptop" ]]; then
    export PATH="$HOME/bin:$PATH"
fi
```

[16] http://ezprompt.net/

[17] https://github.com/powerline/powerline

[18] https://ohmybash.nntoan.com/

[19] https://github.com/Bash-it/bash-it

[20] https://ohmyz.sh/

Using CLI For Specific Tasks

Accounts and Permissions

We can manipulate system user accounts, switch between them, run programs under different users, and control permissions within a command line. On Unix-like systems, users also belong to one or more groups. Therefore permissions can be set per group, rather than just per user. The primary programs for managing users and groups are listed below.

Utility	Description
`useradd`	Add new users
`userdel`	Remove users
`groupadd`	Add new groups
`groupdel`	Remove groups
`usermod`	Add users to groups
`passwd`	Change passwords for users

It is necessary to have special privileges to perform all these actions, often done by performing such actions as *root* users. A root user is a superuser. It is essentially an administrator's account. For practical and security reasons, other accounts are usually used for administration instead, with the ability to run commands under a root user with the `sudo` command.

```
> useradd -m newuser ①
useradd: Permission denied. ②
useradd: cannot lock /etc/passwd; try again later.
> sudo useradd -m newuser ③
```

① Creating a new user. `-m` option will also create a home directory for this user.

② We can't create a user with the current permissions.

③ By prefixing the command with `sudo` and entering the correct root password, the system allows us to run the command. For this to work, our user needs to be in a special *sudo group* (the concrete name differs based on the OS).

Whenever such error occurs due to running a command under a user other than root, we can type `sudo !!`, and the shell will expand the previously entered command for us.

It is also possible to use the utility `su` to switch to the root user permanently, although this is not usually necessary. Both `sudo` and `su` are for running commands under any other user. If we don't specify the user, they will assume the command should be run under `root`.

Before we move on, let's see how to check if a user is a part of a particular group:

```
> id username ①
uid=1000(username) gid=1000(username)
groups=1000(username),10(wheel),972(docker) ②
```

① The `id` utility displays basic information about users. If no arguments are provided, `id` shows the information about the user running the shell.

② The user is part of three groups, one of them being `username`, since each user has its group as well. `wheel` is a standard group on Fedora systems that allows users to use `sudo`.

File Permission System

Linux/Unix permission system is based on *file permissions*, since all resources are accessible in a virtual file system where everything appears as files. We can always examine the file and folder permissions with `ls -l`. Each file and folder has a user owner, a group owner, and a set of permissions. The first column of the output displays the permissions, while the group and user owners appear later.

```
> ls -l
drwxr-xr-x.  2 group user 4096 Dec 12 15:05 dir1
-rw-r--r--.  1 group user    0 Dec 12 15:05 file1.txt
-rw-r--r--.  1 group user    0 Dec 12 15:05 file2.pdf
```

The first character on the line denotes the file type, `d` for directory and `-` for a regular file. The permission output follows after that. It can be divided into three groups (each group having three characters):

- Permissions of the user that owns the file or the directory
- Permissions for the group the user performing the operation would need to be part of
- Permissions for other users and groups

These three groups decide who can do what with the file or the folder on the system.

The output uses characters like `r` for reading, `w` for writing, and `x` for executing files. The combination of these operations can also be represented by a single number, called *octal notation*. We will use the proper number later to change the permissions.

Allowed operations	**Number**
Nothing (—)	0
Execute (–x)	1
Write (-w-)	2
Read (r–)	4
Write and execute (-wx)	3
Read and execute (r-x)	5
Read and write (rw-)	6
Read, write, and execute (rwx)	7

When we put together the proper numbers for the user owner, group owner and other users, we will arrive at a three digit number, like `400` for `r--------` or `777` for `rwxrwxrwx`.

Changing Ownership and Permissions

The owner of files can be changed with the utility `chown` (change owner). Imagine a situation when we might want a web server to be the owner of some static assets that it should serve over HTTP. In that case, it is advantageous if the webserver has its user to set the permissions exactly as needed. Once users and groups are set, the permissions set by `chmod` (change mode) will control the allowed operations. Let's see how to apply it in practice.

```
> ls -l
-rw-r--r--.  1 group user    0 Mar  8 17:23 file1 ①
> chown webserveruser file1.txt ②
> chown :somegroup file1.txt ③
> ls -l
-rw-r--r--.  1 somegroup webserveruser    0 Mar  8 17:23 file1 ④
> chmod 400 file1.txt ⑤
> ls -l
-r--------. 1 somegroup webserveruser    0 Mar  8 17:23 file1
```

① The `file1` is owned by the `group` group and `user` user, allowing read and write access for the owner and read access for everyone else.

② The owner is changed to `webserveruser`.

③ The group owner is changed to `somegroup`.

④ The changes are reflected in the `ls -l` output.

⑤ Now, we redefine the allowed operations. Only the user owner, in our case `webserveruser`, will have the read access. All other users won't be able to do anything with the file.

There are other ways to change the permissions than using the octal notation, as we will see in the section for writing Bash scripts.

Package Management

Operating systems and programming languages feature package managers to install programs and software libraries. We can use them to install software on workstations and servers programmatically. Usually, the software would be packaged and available for our operating system, but sometimes we need to use a different package manager if the application is not available there. For example, the program `thefuck` is written in Python and can be installed with `pip install thefuck` if it is not in the package manager of the operating system. Of course, the package manager `pip` would have to be installed first.

To install software, we will typically need to use `sudo`, as in `sudo dnf install <package>` on Fedora. Some package managers like pip make it possible to install packages only for the

current user without sudo, e.g., `pip install --user <package>` instead of performing the system-wide installation with `sudo pip install <package>`.

Table 6. Examples of package managers

Name	Description
dnf[21]	Package manager for Fedora and Red Hat operating systems
apt[22]	Package manager for Debian and Ubuntu operating systems
brew[23]	Package manager for macOS operating system
npm[24]	Package manager for JavaScript
pip[25]	Package manager for Python

Some software might not be distributed through any package manager. In that case, we might need to download it directly or sometimes even build it from the source code.

Job Control

When a program is executed on a command line, it will write its output directly to the terminal. We are then unable to issue other commands or interact with the command line until it finishes. We say that such commands run in the *foreground* and call them *jobs*. We can also run a program in the *background* if we wish to, detaching its output from the terminal. This detachment is achieved by appending `&` character when issuing commands. Let's see how this works with the program `sleep` that will simulate a long-running command:

```
> sleep 20000 &
[1] 11774
>
```

Once a program is started in the background, we can immediately use the command line again. We also receive two numbers that can be used to control the job. The first is a simple reference number that will allow us to reference the job in the terminal session

with `%X` like `%1` and a *pid*, a system process id.

We can use the `jobs` command to get the list of all unfinished jobs started in a shell session. In this case, it will print information about the sleep program running in the background (to see also the process ids, pass the `-p` option):

```
> jobs
[1]  + running    sleep 20000
> jobs -p
[1]  + 11774 running    sleep 20000
```

We can bring any program running in the background back to the foreground with `fg`, using the job reference number:

```
> fg %1
```

The process can be paused with `Ctrl` + `Z` and resumed later in foreground or background with `fg %1` or `bg %1`.

Every process in a Unix-like operating system can be controlled by sending *signals*. The process id (*PID*) will identify the process. To terminate, suspend or resume a process, we will use a program called `kill`. The basic usage is `kill -SIGNAL <pid>` where *SIGNAL* can be the signal identifier or its number.

Table 7. The most important signals for terminating, suspending and resuming processes

Signal (number)	**Explanation**
SIGINT, SIGTERM (15)	Interrupts/terminates a process gracefully. The program is allowed to handle the signal. `Ctrl` + `C` generates SIGTERM for the running process. This shortcut is often used on the command line to stop a running command.
SIGSTOP (19)	Suspends a process which can be resumed later. `Ctrl` + `Z` generates SIGSTOP.
SIGCONT (18)	Resumes a suspended process.

Signal (number)	Explanation
SIGQUIT (3)	Terminates a process, cleans up the resources used, and generates a core dump. `Ctrl` + `\` generates SIGQUIT.
SIGKILL (9)	Kills the process without the cooperation with the program. It is typically used when the program is not responding to other signals.

Let's see it in action on our process, assuming it is currently suspended in the background:

```
> kill -SIGCONT 11774 ①
> kill -9 11774 ②
> jobs ③
>
```

① The program will be resumed in the background.

② The program execution is ended with the signal number 9 (we could have used `-SIGKILL` instead).

③ After the process is killed, the `jobs` program won't output anything since there are no more unfinished jobs started in the session.

Finding the Process Id

It is also possible to control a process that wasn't started in the current terminal session. The program `top` (table of processes) can be used to list running processes and their ids:

```
> top
top - 16:09:30 up  4:30,  1 user,  load average: 0.69, 0.85, 0.73
Tasks: 307 total,   2 running, 305 sleeping,   0 stopped,   0 zombie
%Cpu(s):  2.0 us,  1.8 sy,  0.0 ni, 95.5 id,  0.3 wa,  0.3 hi,  0.2 si,
0.0 st
MiB Mem :  15819.2 total,   6701.9 free,   4542.2 used,   4575.1
buff/cache
MiB Swap:   7988.0 total,   7988.0 free,      0.0 used.  10567.8 avail
Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+
COMMAND
   2492 pstribny  20   0 3356388 243780 132092 S  17.6   1.5   6:10.13
gnome-shell
   6520 pstribny  20   0  632084  81636  64916 S  10.3   0.5   0:52.28
tilix
   3247 pstribny  20   0 3930492 515908 159928 S   1.0   3.2  14:53.14
firefox
  ...
```

A typical situation might be a process blocking a port that we want to use for something else. For instance, we want to run a development web server on port 80, but the port is already taken, perhaps because we already started it before. We can determine which process occupies the port with the command `lsof` (`lsof -i :80`). The process can then be killed using the process id, effectively freeing up the port for another use. To do that in one go, we can use the *command substitution*:

Using kill and lsof together to free a port

```
> kill $(lsof -t -i:80) ①
```

① `-t` option will switch to terse output that shows only process identifiers. The command substitution `$( )` will replace the command with its result, making it

an argument to `kill`. Since it wasn't specified which signal should be sent, `kill` will use `SIGTERM` by default.

System Monitoring

Top is a system monitor that prints information about running processes and their CPU and memory usage so that we can see what is happening on a system. Pressing `H` when `top` is running will display instructions on how to customize its output or use it to kill processes.

While `top` is designed for interactive use, `ps -e` (process status) can be used to get an overview in a single snapshot of the system. Because of that, `ps` is a good alternative to use in scripts.

Instead of using this standard utility, we can install a more powerful alternative called htop[26] with colorized output. If the information displayed is not enough, programs like gtop[27], glances[28] and bpytop[29] display additional monitoring information like disk usage or network statistics. WTF[30] goes even further, making it possible to create a terminal dashboard bringing information about practically anything via installable modules.

Sampler[31] provides monitoring to observe changes in a system by running predefined commands. So if we want to see how the execution of a particular script, program, or process affects the system, Sampler is a good choice.

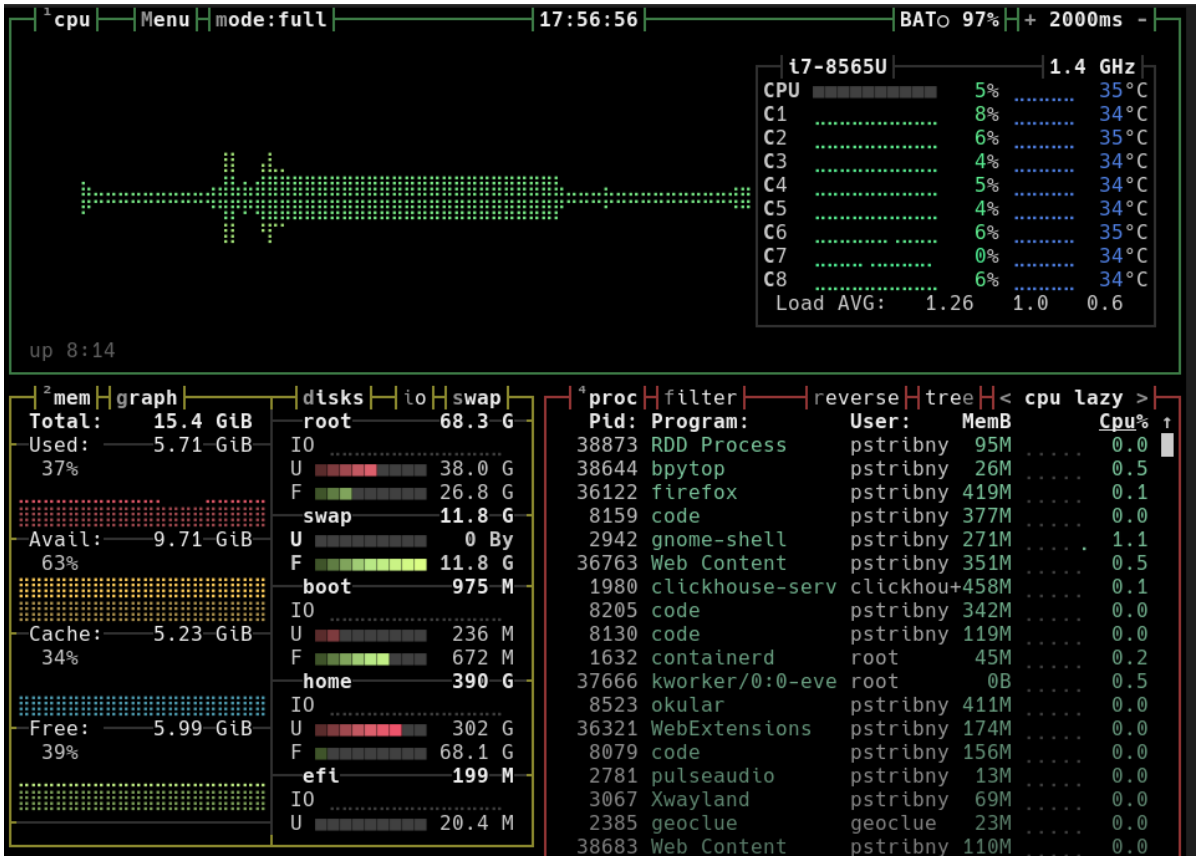


Figure 1. System monitoring with bpytop.

Faster Navigation

Zsh users can leverage “auto cd” functionality to switch working directories by only typing their paths. If a directory name doesn’t collide with anything (e.g., a built-in or function), Zsh will switch to the directory without the need to type `cd` in front of it.

While `cd` can change working directories, it requires a valid path. But often, we need to jump between frequently-used folders. A great little utility Z[32] tracks all places we use and can jump to the most likely place without typing the full path. So if a project “Alpha” is located in `~/work/projects/alpha`, all we would need to type is `z alpha`. Multiple arguments can be provided to solve possible ambiguities, e.g., `z proj alp`.

Z will also output statistics on the most used directories. By typing just `z`, the program

will output all tracked directories together with a relative number of their usage. fz[33] extends Z with tab completion in case we want to combine typing with a graphical selection of the destination.

autojump[34] is an alternative to Z. One additional feature of autojump is the ability to open a file browser in the directory instead of jumping to it.

Search

Searching for Files

The standard programs to search for files by their file names are `find` and `locate`. The main difference between them is that `locate` is much faster but can be outdated as it uses an internal database that has to be regularly reindexed. fd[35] is an improvement over find that has sane defaults and uses colorized output.

Command-line fuzzy finder fzf[36] is an interactive tool that can be used to find not only files but also search within any list that can be provided on the standard input. For instance, we can get an interactive search for our history with `history | fzf`.

We can tell fzf to preview files using our chosen tool with `--preview`, e.g. `fzf --preview 'cat {}'` will display any selected result using `cat`.

Searching within Files

Many times, searching files by their file name is not enough. That's where standard `grep` and its alternatives ripgrep[37] and ripgrep-all[38] come handy. They search for patterns inside files to find the exact matching lines.

Grep searches files (or the standard input) by patterns. It will print each line that matches the pattern, along with the name of the file where it was found. By default, the pattern is assumed to be a regular expression. Positional arguments after the pattern tell grep which files or directories to search.

Grep examples

```
> grep -r '\# [A-Z]' ~/work ~/uni ①
/home/username/work/cli.md:## Terminal
/home/username/work/cli.md:### Shells
...
> grep -Ern 'def |class ' . ②
./app/models.py:8:class User(db.Model):
./app/models.py:22:    def __init__(self, username, email, password,
created_on):
...
```

① Assuming there is a bunch of Markdown notes in folders `work` and `uni`, this prints all headings (lines where a capital letter follows `#`). `-r` is for recursive search to search within folders.

② We search the current directory for all Python function and class definitions. `-n` prints the line numbers, while `-E` makes it possible to use extended regular expressions.

Searching the Standard Input

Grep is often used to perform a search on the standard input. Typically, text lines are collected from the output of a program and passed to grep using the pipe operator.

```
> history | grep -w grep ①
  320  grep -Ern 'def |class ' .
  321  history | grep -w lsof
...
> compgen -c | grep mysql ②
mysqlcheck
mysql_install_db
mysql_upgrade
mysql_plugin
mysqldump
...
```

① We search the command history for all commands that were executed before. `-w` will match only exact words so that only commands that have a standalone "grep" string are matched. If we don't remember what we did, this is an excellent way to recover some lost knowledge.

② The `compgen` utility lists possible command auto-complete matches. We can use its database to find executables with a particular name. In this case, we look for all utilities that have "mysql" in the name.

Ripgrep is a fast alternative to grep with better defaults. It automatically shows line numbers, ignores files from `.gitignore` files, and skips hidden and binary files. It can also search compressed files.

Ripgrep usage

```
> rg PATTERN [PATH] ①
> rg -F EXACT_STRING ②
> rg -tpy PATTERN ③
> rg --hidden PATTERN ④
> rg --sort path PATTERN ⑤
> rg --context X PATTERN ⑥
```

① Ripgrep is invoked with `rg` and will automatically search for a regex pattern in the current working directory unless an optional path is provided.

② Exact strings can be matched the same way as in grep with `-F`. This is useful when we need to use characters like `(`, `)`, `[`, `]`, `$` that have special meaning in regular expressions.

③ `-t` option can be used to restrict the search to a particular type of file. In this case, the search will be performed only in Python files. Capital `-T` will do the opposite: it will exclude a specific file extension from the search.

④ To search inside hidden files, use the `--hidden` argument.

⑤ Ripgrep uses parallelism to achieve better speed. If we need a consistent output (for instance, when we want to process it by another tool), we can add `--sort path` option to make the order consistent. This option will, however, disable parallelism.

⑥ We can extend the output of Ripgrep to show surrounding lines with `--context X`, e.g., `--context 20` would show 20 lines around the matched term.

It is a good idea to put a common flag configuration to a `~/.ripgreprc` config file. For instance, a color configuration can be placed here if we don't like the defaults.

For searching within special file types, use ripgrep-all. It can read many different files like PDFs, e-books, office documents, zip files, and even SQLite database files.

Searching Codebases

There are some search utilities optimized for search in source files. ack[39] is a grep-like tool designed for programmers. It automatically ignores directories like `.git`, skips binary files, can restrict the search to a certain language with arguments like `--python` and so on. An even faster alternative is The Silver Searcher[40] (run with `ag`). Besides being

faster, it can also pick up patterns specified in version control files like `.gitignore` and allows for custom ignore patterns to be stored in a file.

Comparison of various text and code search tools can be found on Feature comparison of ack, ag, git-grep, GNU grep and ripgrep[41] page.

Disk Usage

There are situations when we need to extract information about the disk space on a command line. For instance, if a server is not responding to requests, it might be because its disk is out of space. Although there are better ways to monitor such things at scale, it is always good to know how to quickly check the disk usage.

To get a nice colorized report of disk space available and used on mounted file systems, we can use pydf[42] which is a variation of the more standard utility `df`. The same information, but with even nicer output, can be displayed by duf[43]. These programs run fast but only display information from the used disk blocks in the file system metadata. What if we are interested in how much space the folders and files take inside a file system?

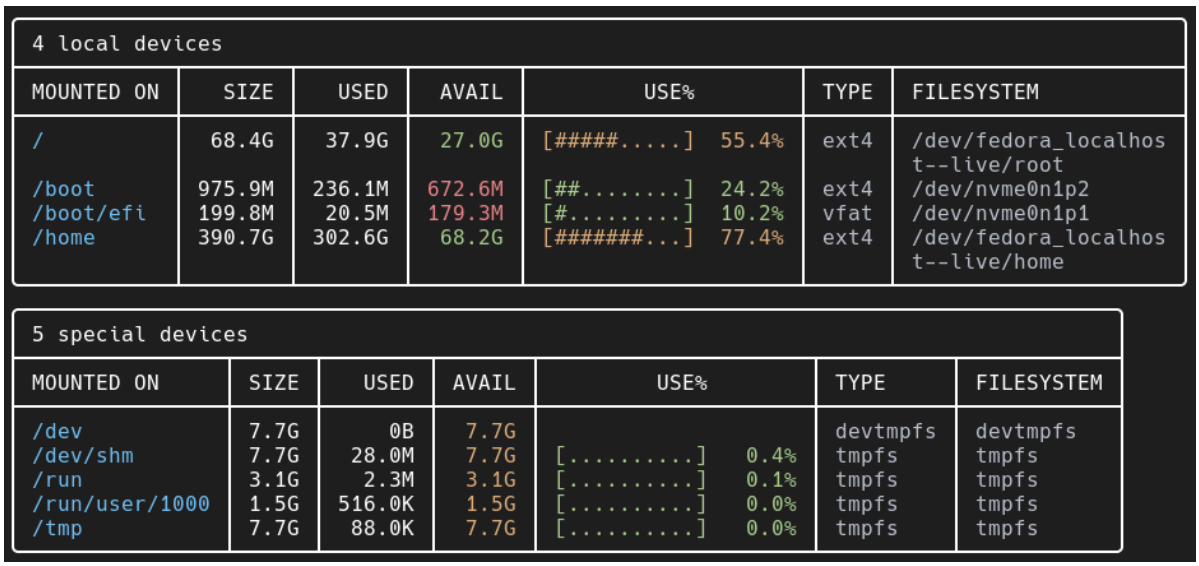


Figure 2. Colorized output of duf, disk usage utility.

To quickly check disk usage in a particular folder, we can use the utility called `du` (disk usage). It will go through the directory tree, examining the sizes of files and summing

them up. Run it with `du -h --max-depth=1` to see the overview of just a single directory with human-readable file sizes. Even better than `du` is ncdu[44] (NCurses Disk Usage). It allows comfortable, interactive browsing through the file system, displaying the file and folder sizes in a readable graphical way.

File Management

New directories can be created with `mkdir`. `mkdir -p` can even create a folder in a non-existing location. This option is useful when we want to nest the new folder under a structure that doesn't exist yet. The opposite, removing a directory, can be achieved with `rmdir`. Both programs can take multiple directory paths as their positional arguments. Files and directories can be moved and renamed with `mv`, and copied with `cp`.

fcp[45] is a `cp` alternative for faster copy operations that can be made parallel on *Solid State Drives* (SSD). Use it when transferring large files or when copy operations are part of automated scripts that run many times.

To delete files, use `rm`. The most important options are `-i` to prompt for confirmation before deleting files (instrumental when using globbing to prevent accidents), `-r` to delete files recursively, allowing rm also to delete directories, and `-f` never to prompt the user about anything. rip[46] is a safer rm alternative that doesn't delete files immediately but rather moves them to a dedicated directory for later recovery.

There are multiple interactive and non-interactive alternatives to `ls` for exploring directories. Exa[47] and colorls[48] both offer much nicer colorized outputs. Because directories are trees, it can be useful to display them as such. The basic program for that is called `tree`, and we can control how deep the output of it will be with `-L` option, e.g. `tree -L 2`. broot[49] is like `tree`, but interactive and supercharged with additional features.

Working with folders

```
> mkdir -p work/notes ①
> cp -r work school ②
> tree ③
.
├── school
│   └── notes
└── work
    └── notes

4 directories, 0 files
> rm -r work school ④
```

① A new `work` directory is created, with another directory `notes` inside it.

② To copy a directory, `cp` requires `-r` (recursive) option, followed by the source and target directory paths. A new directory `school` is created as a copy of `work`.

③ We check that the new folder structure is how we wanted with `tree`.

④ If we don't like it, we can delete both folders at once.

There are also some other awesome file managers with interactive TUIs besides broot. Midnight Commander[50] is a two-pane file manager. Ranger[51] is a popular file manager with vim-like control, tabs, bookmarks, mouse support, and file previews. nnn[52] is a modern alternative to ranger that features regex search, disk usage analyzer, and other features.

Archiving and Compression

It is common to work with compressed and archived files, usually `.tar` and `.zip` files. TAR stands for *Tape ARchive*, created initially to store files efficiently on magnetic tapes. It is a file format and a utility of the same name that preserves Unix file permissions and the directory structure. The files themselves are often called *tarballs*.

Tar itself is not used for compression but only for creating one file from multiple ones. That's why we often see file types like `.tar.gz`, which refer to files that are archived and compressed afterward with Gzip. Files can be compressed and uncompressed with `gzip` and `gunzip` utilities. Since the archive is typically compressed as one file, it is impossible to modify its contents afterward.

```
> tar -cf myfolder.tar myfolder ①
> gzip myfolder.tar ②
> ls
myfolder  myfolder.tar.gz ③
> rm -r myfolder ④
> gzip -d myfolder.tar.gz ⑤
> gunzip myfolder.tar.gz ⑥
> tar -xf myfolder.tar ⑦
```

① Creating an archive with `tar` requires `-c` to create an archive and `-f` option to write it to a file. The first argument is the name of the desired archive file, and the second argument is a path to files that we want to put into the archive.

② Using `gzip` is simple. Just tell it what file should be compressed.

③ The result is our original folder and the same folder compressed in a tarball.

④ Our folder is archived and compressed for, e.g., a backup, so now we can remove the original folder.

⑤ `gzip` can decompress a file with `-d` option.

⑥ Alternatively, `gunzip` can be used without any options to do the same.

⑦ Finally, we extract the files from the tarball using `-x` option for `tar`.

The process of unpacking a Gzip compressed archive is so common that the `tar` program itself supports doing so in one go with `tar -xvzf myfolder.tar.gz`. The

reverse can be done too with `tar -cvzf myfolder.tar myfolder`.

Modern multi-core and multi-processor machines can take advantage of parallel Gzip compression with pigz[53]. There are similar tools for other formats and/or decompression. Look for them in case the compression is too slow for you.

When we need to work with Zip format, commonly used on Windows systems, we can use programs `zip` and `unzip`. Zip was created for MSDOS, so, unfortunately, Zip doesn't preserve Unix file attributes, e.g., a file won't be marked as executable after decompressing. Zip archives are already compressed, and the files are compressed individually, in contrast to `.tar.gz` files. `zipinfo` command can be used to inspect an archive without decompressing it.

Other compression formats and tools have different tradeoffs regarding the compression and decompression speed, compression size, or the general availability of such tools on different systems. For instance, `xz` is a modern compression tool that is efficient regarding the final file size.

Viewing Files

The basic program for viewing text files is `cat`. One interesting option when using cat is the ability to print line numbers for the output with `cat -n`. A great alternative is a program called bat[54] which displays line numbers automatically and offers beautiful syntax highlighting for common file types. More specialized programs can be installed for working with a specific file type, fx[55] for JSON and glow[56] for Markdown.

When dealing with large text files like logs or data sets, we can peak in to see the first few lines with the program `head`, or the last few lines with `tail` (with `-n NUM` as the option for how many lines to show). Since log files can grow continuously, we might want to keep reading the updating log with `tail --follow` or `tail -f` for short. This way, new log messages are displayed as they come in.

In Linux distributions that use systemd, it is possible to see the logs from running services with `journalctl` utility. `-n NUM` specifies the number of log lines to display, while

`-u unit` can be used to restrict logs only to a particular service. For instance, `journalctl -n 100 -u postgresql` would display 100 last lines of PostgreSQL service logs.

The Log File Navigator[57] can collect logs from a directory and display them together color-highlighted in the terminal. It allows for filtering and querying local log files. We can also slice and dice logs on the command line with angle-grinder[58].

To open a file in a default system application from a command line, use the program `xdg-open`. It is helpful for images, web pages, PDFs, or other types of files that we don't want to look at inside a terminal window.

Editing Text Files

The most famous terminal editors are Vim[59] and Emacs[60] (although Emacs is primarily a desktop editor, it can run in the terminal). They focus on efficient ways of editing text. Both of them work with multiple modes, one for writing and others for editing text using key sequences or for running other commands. If working with Vim or Emacs is too complicated, we can use simpler and more user-friendly Nano[61].

Opening a file for editing is as simple as passing the file path as an argument:

```
> vim ~/notes.txt
> nano ~/notes.txt
> emacs -nw ~/notes.txt ①
```

① Emacs requires special option in order to start inside a terminal window.

Learning Vim

We can learn Vim by using the Vim tutor. It can be run directly on the command line with `vimtutor`. There is also a game, Vim adventures[62], that teaches essential Vim commands.

Text Data Wrangling

Working with text data, both structured and unstructured, is an everyday activity. It is handy to make some repetitive text transformations and edits quickly from the command line. This includes tasks such as:

- Extracting information from log files, including logs from web servers, databases, systemd services, and other applications
- Extracting information from dumps, e.g., from profilers
- Updating text-based configuration files. For instance, updating configuration for cron or a database, replacing placeholders for application deployments, and so on.
- Connecting outputs and inputs of programs when they don't match exactly
- Cleaning data for further processing by manipulating CSV, JSON, and other formats
- Comparing files to each other (making diffs)

Working with Sed

Sed is, from its name, a stream editor. It allows us to find and then replace, insert, or delete text from the standard input or a file and put the result to the standard output. If the input is a file, sed can also change it in place.

Practical Use of Sed

By default, sed processes text input line by line and performs a specified operation on the text matched by a regular expression. The basic operations are:

- `s` for replacing the matched string
- `c` for replacing the whole line
- `a` for adding a line before the matched search
- `i` for adding a line after the matched search
- `d` for deleting lines

Although we will go through a couple of examples, complete documentation can be found in man pages or online in the sed manual[63].

Replacing Text

We will build upon our previous example of replacing text, but this time we will modify an existing file. The syntax for the replacement operation is `s/pattern/replacement/flags`.

`s` is the operation, `pattern` is the regex to match the string we are looking for, while `flags` control the behavior. Let's now modify a configuration file `.env` with placeholders that we want to replace:

.env file

```
USER_NAME=USER_PLACEHOLDER
USER_PASSWD=USER_PASSWD_PLACEHOLDER
```

Replacing placeholders in a file

```
> sed -i 's/USER_PLACEHOLDER/petr/' .env ①
> sed -i -e 's/USER_PLACEHOLDER/petr/;
s/USER_PASSWD_PLACEHOLDER/passwd/' .env ②
> sed 's/USER_PLACEHOLDER/petr/' .env > .env2 ③
```

① `-i` option can be used to replace a file in place. If we specify a file or files at the end, sed will read them instead of reading the standard input.

② We can perform multiple operations at once by using `-e` option. The operations have to be separated with a semicolon. In this case, we replace both placeholders.

③ Instead of modifying the original file, we redirect the output to the file `.env2`.

One helpful flag is `g`, meaning global. By default, sed will only replace the first occurrence of a match at every line, so we have to use `g` to replace all matches. To replace the word "Unix" with "Linux", the command would look like `echo "Unix is the best OS. I love Unix!" | sed 's/Unix/Linux/g'`.

In the following example, we will look at how to replace tabs with spaces to settle the war between the two once and for all:

```
> sed 's/\t/    /g' tabs.txt > spaces.txt ①
```

① Sed will replace all tabs with four spaces in the file `tabs.txt`. We redirect the output to a new file with `> spaces.txt`.

Sed and Regular Expressions

So far, we have always replaced a plain string with another plain string. The pattern, however, can be a regular expression capturing groups, while the replacement string can include the matched parts. To demonstrate that, we will work with a system log from the PostgreSQL service, saved into `postgres.log` file:

postgres.log

```
Dec 13 19:40:40 laptop systemd[1]: Starting PostgreSQL database
server...
Dec 13 19:40:40 laptop systemd[1]: Started PostgreSQL database server.
Dec 14 00:24:53 laptop systemd[1]: Stopping PostgreSQL database
server...
Dec 14 00:24:53 laptop systemd[1]: postgresql.service: Killing process
1629 (postmaster) with signal SIGKILL.
Dec 14 00:24:53 laptop systemd[1]: postgresql.service: Succeeded.
Dec 14 00:24:53 laptop systemd[1]: Stopped PostgreSQL database server.
Dec 14 00:24:53 laptop systemd[1]: postgresql.service: Consumed 10.578s
CPU time.
```

We will start by extracting the log message by dropping the first part of each line. After that, we will use `bat` to render the message in bold.

```
> sed -e 's/.*\]: \(.*\)$/\1/' postgres.log ①
> sed -e 's/\(.*\)\]: \(.*\)$/\1\]: \*\*\2\*\*/' postgres.log | bat -l
md ②
```

① The regular expression divides the log line into two parts based on the `]:` separator. The parentheses put the second part of the string into a capture group that is later referenced with `\1`. We have to escape the parenthesis for capture groups (`\(` and `\)`) in the default mode. The result is a plain message, without the time and other information.

② Two capture groups are used to generate the line. We place two asterisks around the message, which means bold in Markdown. The output is then fed into `bat`, specifying the Markdown format using the `-l` option. The result can be seen in the picture below.

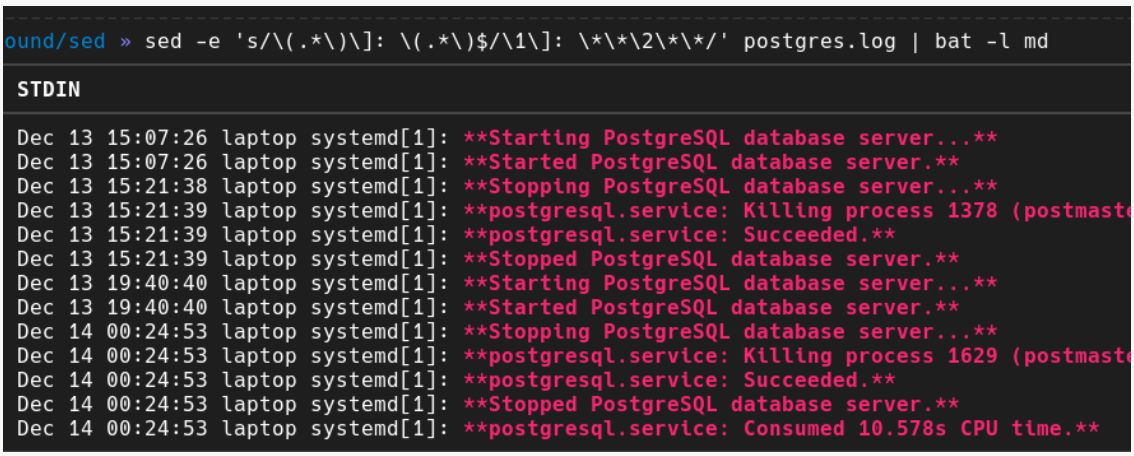


Figure 3. The result of combining sed and bat to render a message as Markdown.

Sed uses POSIX BRE (Basic Regular Expression) syntax. Extended regular expressions can be enabled by `-r` or `-E` option depending on the platform.

Other Sed Operations

Other operations like `c`, `d`, or `a` are written after the matching slash: `/pattern/c`. Let's now introduce the "replace line" operation `c`, implementing the same placeholder replacement as before. Because we are replacing the whole line, we need to specify

how the whole new line should look like:

```
> sed -i '/USER_PLACEHOLDER/USER_NAME=petr/c' .env
```

Inserting lines with `a` and `i` operations follow the same syntax, only the text is appended after or prepended before the match. The delete operation `d` can be useful to clean up a file or hide certain lines:

```
> sed -e '/^#/d' /etc/services | more ①
```

① We remove all comment lines that start with `#`. `/etc/services` and many other Linux/Unix locations and configuration files contain such comments, so the output can be cleared this way. Passing the output to `more` command will paginate the results.

Matching Lines With Separate Patterns

Sed also has the ability to match a line first by a separate pattern, match a line between two patterns, and allows specifying on which lines the operation should occur. We can write `/pattern/`, `/beginpattern/,/endpattern/`, `NUMBER` or `NUMBER,NUMBER` in front of the command to specify the lines we want to process, and sed will only execute the command on those lines.

```
> sed -i '/=/ s/USER_PLACEHOLDER/petr/' .env ①
> sed -i '1 s/USER_PLACEHOLDER/petr/' .env ②
> sed -i '1,2 s/USER_PLACEHOLDER/petr/' .env ③
```

① This is the same placeholder replacement as before, but we tell sed to only make the replacement on lines that contain the equality character (`=`).

② The same, but telling sed to only make the replacement on the first line.

③ The same, but telling sed to only make the replacement on the first two lines.

Changing the Separator

Sometimes we need to use slashes in the sed command, e.g., when working with paths. Instead of escaping all the slashes, we can use another separator like `|` or `_`.

Using a different separator in sed

```
> echo '/usr/local/bin' | sed 's|/usr/local/bin|/home/username/bin|'
/home/username/bin
```

A useful flag for experimenting with sed is `--debug`, which will print some debugging information to explain what is happening. If we have multiple sed commands we want to run at once, we can also store them in a file and load the file with the `-f` option.

Working with Records

Awk is another tool to process text files line by line, but `awk` focuses on working with lines as records, dividing each line into separate records based on a separator. It is also a complete programming language with conditionals, loops, variable assignments, arithmetics, etc. There is no possible space to go into the details of Awk, so we will only look at some examples that are easier done with Awk than sed.

By default, Awk parses each line automatically using any whitespace as a delimiter and exposes the individual records as variables `$1`, `$2`, `$3`, and so on. Note that it is better to always quote Awk script with single quotes because of the dollar symbols so that the shell won't expand them to variables. Leading and trailing whitespace is ignored. Other than that, the simple usage of Awk is like sed: we match lines that we want to work with using a pattern and then perform some action with that line.

A good example might be getting the list of Linux users from `/etc/passwd`. This file lists each user per line, together with some additional information where each record is separated by a colon. The user name is at the beginning:

/etc/passwd

```
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
pstribny:x:1000:1000:Petr Stribny:/home/pstribny:/usr/bin/zsh
```

Processing /etc/passwd with Awk

```
> awk -F':' '{print $1}' /etc/passwd ①
root ②
bin
daemon
pstribny
> awk -F':' '$7 == "/usr/bin/zsh" {print $1}' /etc/passwd ③
pstribny
```

① We need to change the default separator from whitespace to a colon using `-F` option. It can take a regular expression, but in this case, our single character `:` is all we need. The block `{ }` is the script that should be executed on each line. The first record will be parsed and available automatically via `$1` and printed.

② The result is the list of all users on separate lines.

③ We can limit what gets processed by a pattern or a conditional. The last part of `/etc/passwd` file points to the shell the user is using. So, in this case, we get all user names of people who have their shell set to Zsh.

While working with CSV files in Awk is tempting, parsing CSV files is much more difficult as the separators are not always clear (the same character might appear escaped or quoted inside records). For CSV it is best to use specialized tools like xsv[64] and csvkit[65].

A Note on Regular Expressions

We have seen that programs, like grep and sed, operate with two types of regular expressions that are part of the POSIX standard: Basic Regular Expressions (BRE)[66] and Extended Regular Expressions (ERE)[67]. The syntax of regular expressions was standardized to make some basic system utilities consistent.

We can switch between these two options in those programs, typically using `-E` to enable the extended version. Awk behaves differently, using the extended version by default. Note that other programs or libraries might have their implementation, behaving differently.

`cut` is another utility that can perform similar tasks as awk. It is not a complete language

like Awk, but some people might prefer its interface for manipulating record-type lines.

Working with JSON and Other Formats

While sed is helpful for general regex style text manipulation and awk for manipulating tabular data, sometimes we want to work with specific file formats. JSON is one of the most commonplace data exchange formats nowadays, so it can be useful to process and manipulate it on the command line.

The fx tool can be used not only as an interactive and non-interactive JSON viewer, but also as a JSON manipulation tool to extract information or make changes in place. It accepts JavaScript as an argument that can use some fx's built-in functions or even contain third-party library calls. jq[68] is another command-line JSON processor for those that don't like JavaScript. The same, but for YAML is provided by yq[69]. XML can be manipulated with XMLStarlet[70].

Let's say we have a JSON file with the current job offers on the market:

job_offers.json

```
{
  "jobOffers": [
    {
      "position": "Python developer at Snakes, Inc.",
      "language": "Python"
    },
    {
      "position": "Full-stack developer at WeDoEverythingOurselves",
      "language": "JavaScript"
    },
    {
      "position": "Superstar at GalaxyWorks",
      "language": "Python"
    }
  ]
}
```

Using jq, we will extract the popular programming languages people are hiring for:

```
> cat job_offers.json | jq --raw-output '.jobOffers[].language' ①
Python
JavaScript
Python
```

① We feed the JSON file to `jq` using the pipe operator. `--raw-output` will allow us to get a non-JSON string as the result of our query (otherwise, jq builds a JSON object by default - in this case, the strings would have quotes around them). The last part is the jq's internal query language that allowed us to extract the information without much effort.

To explore JSON files interactively instead, ijq[71] offers a nice TUI just for that. It can be thought of as an "interactive jq".

Wrapping Up

Finally, this section would not be complete without mentioning these commonly used utilities for data wrangling: `sort` for sorting text lines, `uniq` for removing duplicates, and `wc` for word and character counts.

```
> cat job_offers.json | jq --raw-output '.jobOffers[].language' | sort | uniq
-c ①
      1 JavaScript  ②
      2 Python
```

① `uniq -c` will merge duplicates and print the number of occurences. It expects that the same lines are next to each other, therefore we need to pass the text through `sort` first.

② We get the counts together with the line string. It seems that Python is in demand.

Summary

- To find and filter information inside files or in the standard input, use grep, ripgrep and their alternatives.
- To search source code, use ack or the Silver Searcher.
- To transform text lines using regex expressions, use sed.
- To transform text lines with records that have a clear separator, use Awk or cut.
- To work with CSV files, use xsv.
- To work with YAML files, use yq.
- To work with XML, use XMLStarlet.
- To extract information from JSON files or to modify them, use jq or fx.
- Combine all the tools together as you see fit.

Basic Networking

`ping` is a famous command to send repeated ICMP[72] echo requests to network hosts. It is used to find out if a remote computer, server, or device is reachable over the network. To use it, follow the ping command with a positional argument representing IP version 4 or 6 address or with a domain name pointing to such address. gping[73] does the same but draws a chart with response times rather than printing the results line by line.

Domain Name System (DNS) can be queried using dog[74]. It is useful for getting information about domain names, e.g., to determine if a performed change in a DNS entry has been propagated already.

HTTP Clients

The most common HTTP client for the command line is without a doubt curl[75], appearing in many documentation and testing examples. Curl comes preinstalled on many systems and understands many protocols, not just HTTP. When we need something

more user-friendly with fancy syntax highlighting, HTTPie[76] is a great curl replacement for many tasks. If we want to explore HTTP APIs interactively, HTTP Prompt[77] from the same author is an interesting alternative. Curlie[78] brings HTTPie-like interface to curl for performance and compatibility reasons.

Let's see how we can check the EUR/USD rate by fetching the rates from an HTTP endpoint and processing it on the command line:

```
> curl https://api.exchangeratesapi.io/latest --silent | jq | grep USD ①
    "USD": 1.1926, ②
```

① `curl` takes the URL as a positional argument. `--silent` option will suppress additional curl output about the transfer, outputting only the response text. Because the JSON is not formatted, we pass the output through `jq`, which will place all JSON fields on separate lines. Finally, we will filter the results by the currency we are interested in.

② We get the line that contains "USD". We know that we can exchange 1 EUR for 1.1926 dollars.

The wget[79] utility can be used to download files, pages, and whole websites since it can follow links on web pages. It is a simple web crawling tool.

SSH and SCP

SSH stands for *Secure Shell*. It is a protocol that we can use to operate remote machines. It is a client-server protocol, meaning that the remote machine has to be configured to support it, while we need one of many client-side applications that can speak the protocol. SSH supports executing commands remotely, port forwarding, file transfer, and other things. The most important applications that we might want to use are `ssh` for executing commands and `scp` or `rsync` for file transfers. We will see them in action shortly.

The authentication with the server can be handled with just a password or a private-public key pair. When we create a new server instance (using a cloud provider like Digital Ocean or AWS), we will be given a machine with a root user and password. Some providers like Digital Ocean can insert our public key on the server automatically. In other instances, we will have to log in to the machine and place our key in `~/.ssh/authorized_keys` file, which is where authorization keys are typically stored in

Unix-like systems. We can also use `ssh-copy-id` utility to copy the key for us. Once our key is configured on the remote server and we can authenticate using it, we might want to turn off password-based authentication to increase security.

Creating a Private-Public Key Pair

If we don't have our cryptographic key pair yet, we can generate one using a utility called `ssh-keygen`. There are two essential choices to make. The first one is to decide what crypto algorithm we want to use to sign and verify the keys. Today, the most common ones are RSA, with a recommended size of at least 3072 bits, and Ed25519, with a length of at least 256 or 512 bits.

The second one is the choice of whether we want to have a passphrase associated with our private key, effectively protecting it with a password. It makes it more difficult for other parties to use our key when stolen. Some people prefer not to set a passphrase because it makes using the key simpler. But we can set a passphrase and avoid having to type the key password again and again on every use by utilizing `ssh-agent` or `gpg-agent` utilities. This approach gives us a more secure private key but doesn't hinder everyday interactions with SSH-based tools.

How to generate Ed25519 key pair with ssh-keygen

```
> ssh-keygen -o -a 100 -t ed25519 -f ~/.ssh/id_ed25519
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/username/.ssh/id_ed25519.
Your public key has been saved in /home/username/.ssh/id_ed25519.pub.
The key fingerprint is:
SHA256:PNGnePjvoVL/nVDPq9HrBsF/tYo5HxVoull+FK3LMPs username@hostname
...
```

Check the man page `man ssh-keygen` for details.

Based on the type of key we decided to use, we should have our key pair stored as `~/.ssh/id_rsa` (the private RSA key) and `~/.ssh/id_rsa.pub` (the public RSA key), or `~/.ssh/id_ed25519` (the private Ed25519 key) and `~/.ssh/id_ed25519.pub` (the public Ed25519 key). The content of the `*.pub` files is what we need to get in the

`~/.ssh/authorized_keys` on the server. The `~/` location has to match our intended remote user, e.g. `/home/root`.

Let's now see how we can log into a remote machine using `ssh` and execute commands in a remote session:

Connecting to a server on 10.5.6.22 as username via SSH

```
> ssh username@10.5.6.22 ①
username@server> ②
```

① If our key is not recognized on the server, we will be asked for a password. If we generated a private key with a passphrase, we would have to enter that passphrase now.

② If the authentication succeeds, we are given access to a remote shell session to execute commands on the remote machine. Now is the time when we might want to employ job control techniques or terminal multiplexers to run long-running programs so that we don't have to maintain a permanent SSH connection.

Another way is to execute remote commands in our local shell session. We can do so by simply placing the command as another positional argument to `ssh`:

```
> ssh username@10.5.6.22 "ls ~/"
```

Note that we need to quote the command if there are any spaces, otherwise it would not be recognized as one positional argument. We can also issue multiple commands at once and use other techniques that we saw earlier, like piping.

SSH Configuration

On our local machine, SSH is configured using `~/.ssh/config` file. Here we can predefine our remote connections. This file is useful for aliasing remote servers or specifying various configurations about the remote servers, e.g., when we need to use different key pairs or need to establish a tunnel connection through a bastion host (a server that can connect to other servers to prevent internal servers from being exposed to the internet).

SSH configuration example

```
Host myserver ①
       Hostname 10.5.6.22
       User username
       IdentitiesOnly yes ②
       IdentityFile ~/.ssh/id_ed25519 ③
Include work_hosts/* ④
```

① We name the remote server. We can then refer to it when using SSH-based tools with that name, e.g., `ssh myserver` instead of `ssh username@10.5.6.22`.

② IdentitiesOnly will only send our specified identity (`~/.ssh/id_ed25519`) to the server for authentication. If not set, SSH will send all our local identities for the server to pick the correct one.

③ This is the path to our private key that we want to use to authenticate with this remote server.

④ We can use `Include` to include other files in the configuration. This directive is useful when we want to source the configuration from our dotfiles elsewhere or keep hosts configurations separate in individual files. Globbing is utilized to include multiple matching files, in this case, all files found in the `work_hosts` directory.

File Transfers

Copying files from and to servers is very useful for managing configuration files or doing backups. `scp` is a program for secure file transfers over SSH. Let's have a look at how to use it to transfer a local file to a remote host:

Transfering files with SCP

```
> ls
dir1 file1.txt ①
> scp file1 username@10.5.6.22:~/file1 ②
file1.txt        100%    0     0.0KB/s   00:00 ③
> scp -r dir1 myserver:~/dir1 ④
> scp username@10.5.6.22:~/file1 file1 ⑤
```

① Let's assume we have a file `file1.txt` and a directory `dir1` that we want to move to our remote server.

② `scp` expects the source location first, followed by the target location. We use `username@10.5.6.22` to specify our target server, the same as with SSH. The target file path is separated by a colon.

③ SCP will output basic statistics about the state of the transfer. This was a transfer of an empty file that was very quick, hence the zeros.

④ To transfer directories, we need to use `-r` (recursive) option, as with many other standard commands. In this case, we are referring to our server as `myserver`, thanks to the SSH configuration we did before.

⑤ To download the file back from the server, we just need to switch the arguments so that the remote location is our source path.

`rsync` is a great file transfer utility that improves upon scp by copying only necessary files to perform a synchronization. The basic usage is similar to `scp`, but it can be enhanced with `rsync -av --progress`. This way rsync will preserve things like file permissions and output more details during the transfer.

`rsync` can be used locally as well. It is possible to synchronize directories on a local file system for backups or use it on the remote server to synchronize a Git repository with a deployment folder.

[21] https://en.wikipedia.org/wiki/DNF_(software)
[22] https://en.wikipedia.org/wiki/APT_(software)
[23] https://brew.sh/
[24] https://www.npmjs.com/
[25] https://pip.pypa.io/en/stable/
[26] https://htop.dev/
[27] https://github.com/aksakalli/gtop
[28] https://nicolargo.github.io/glances/
[29] https://github.com/aristocratos/bpytop
[30] https://wtfutil.com/
[31] https://github.com/sqshq/sampler
[32] https://github.com/rupa/z
[33] https://github.com/changyuheng/fz
[34] https://github.com/wting/autojump
[35] https://github.com/sharkdp/fd
[36] https://github.com/junegunn/fzf
[37] https://github.com/BurntSushi/ripgrep/#quick-examples-comparing-tools
[38] https://github.com/phiresky/ripgrep-all
[39] https://beyondgrep.com/

[40] https://github.com/ggreer/the_silver_searcher
[41] https://beyondgrep.com/feature-comparison/
[42] https://linux.die.net/man/1/pydf
[43] https://github.com/muesli/duf
[44] https://dev.yorhel.nl/ncdu
[45] https://github.com/Svetlitski/fcp
[46] https://github.com/nivekuil/rip
[47] https://the.exa.website/
[48] https://github.com/athityakumar/colorls
[49] https://github.com/Canop/broot
[50] https://midnight-commander.org/
[51] https://github.com/ranger/ranger
[52] https://github.com/jarun/nnn
[53] https://zlib.net/pigz/
[54] https://github.com/sharkdp/bat
[55] https://github.com/antonmedv/fx
[56] https://github.com/charmbracelet/glow
[57] http://lnav.org/
[58] https://github.com/rcoh/angle-grinder
[59] https://www.vim.org
[60] https://www.gnu.org/software/emacs/
[61] https://www.nano-editor.org/
[62] https://vim-adventures.com/
[63] https://www.gnu.org/software/sed/manual/sed.html
[64] https://github.com/BurntSushi/xsv
[65] https://github.com/wireservice/csvkit
[66] https://en.wikibooks.org/wiki/Regular_Expressions/POSIX_Basic_Regular_Expressions
[67] https://en.wikibooks.org/wiki/Regular_Expressions/POSIX-Extended_Regular_Expressions
[68] https://stedolan.github.io/jq/
[69] https://mikefarah.gitbook.io/yq/
[70] http://xmlstar.sourceforge.net/overview.php
[71] https://sr.ht/~gpanders/ijq/
[72] https://en.wikipedia.org/wiki/Internet_Control_Message_Protocol
[73] https://github.com/orf/gping
[74] https://github.com/ogham/dog
[75] https://curl.se/
[76] https://httpie.io/
[77] https://http-prompt.com/
[78] https://curlie.io/
[79] https://www.gnu.org/software/wget/manual/wget.html

Writing Shell Scripts

A shell script is a type of executable text file that is understood by the shell. It can execute just one command or group many commands with conditions, loops, functions, and other control mechanisms.

We use shell scripts as installation and configuration scripts that set up an environment for programs to run, as startup scripts that are executed after the computer is started, and for automating tasks where it would be tedious and error-prone to type the sequence of commands manually.

To avoid problems with compatibility between different shells, we will only discuss writing Bash scripts. Even if we use a different interactive shell on the command line, there is no problem in running scripts written for a different shell, as we will see shortly.

Our First Shell Script

To create a script, we need to:

- Tell Bash and other shells how to run our text file using a special first line
- Change the file permissions to make the file executable

Let's create a simple script `print.sh` that prints text to the standard output (the file extension is optional):

print.sh

```
#!/bin/bash ①
echo "Robots are humans' best friends." ②
```

① The first two characters (`#!`) at the beginning are called *shebang*. Following the shebang, we specify which program should execute our file. It is important because we might want to run the script from Zsh or another shell.

② We can put any valid Bash code in the file.

Before we can run it, we need to make this text file executable. We will do it with the command called `chmod`, as described in the *Accounts and Permissions* section. This time, we will not set all permissions at once but only modify them.

```
> chmod u+x print.sh ①
```

① We only allow this file to be executed by the owner of the file with `u+x`. We could also make the file executable by anyone with just `+x`.

To run the script, all we need to do is to type its path:

```
> ./print.sh ①
Robots are humans' best friends.
> bash ./print.sh ②
Robots are humans' best friends.
```

① We prefix our program with a relative path to the current directory (`./`). This way, the shell executes our script and not another program named `print.sh` that might be on the PATH.

② Alternatively, we can pass the script path as an argument to `bash`. In this case, using the shebang inside the script is not required.

Programming in Bash

We have already seen some aspects of Bash programming like the use of variables, short-circuiting operators like `&&`, or conditionals with `if ⋯ fi`. We will now expand this knowledge and see how to create Bash scripts in more detail.

Command and Process Substitutions

Substituting a command with its result is done with `$(CMD)`. This syntax makes it possible to use the command's standard output in a string or as an argument for another command. If we need a file path as the argument, we can use the *process substitution* by writing `<(CMD)`. It will run the command, place its standard output in a temporary file and return the location to the file.

```
> echo "You are now in the directory $(pwd)"
You are now in the directory /output/of/pwd/command
> diff <(ls /location/to/firstfolder) <(ls /location/to/secondfolder) ①
... ②
> echo <(ls /location/to/firstfolder)
/proc/self/fd/12 ③
```

① The `diff` utility can compare files and print their differences. If we pass it the result of `ls` commands, we can compare the contents of directories.

② The output is omitted for brevity.

③ We can use `echo` to demonstrate that the process substitution returns a path to the temporary file.

Comparing files to each other and examining their diffs is useful in general. Try icdiff[80] as an alternative to `diff`, which uses two-column colored output by default.

Functions

Bash functions make it easy to group commands under one name. They can be defined and called inside our scripts as well as on the command line. They are in some ways similar to aliases, as we will see in the next example. The following alias `homedir` and the function `homedir` do the same:

```
# inside a Bash script, e.g. inside a shell startup script

# alias
alias homedir="cd ~/ ; ls"

# function defined on multiple lines
homedir () { ①
    cd ~/ ②
    ls
}

# function defined on one line
homedir () { cd ~/; ls; } ③
```

① Note the spaces around the parentheses and the curly braces.

② We can place commands one after another on new lines inside the function body.

③ If we want to define a function on one line, we need to separate commands with a semicolon, and also place a semicolon after the last command.

The function will be defined and available in our shell session once we execute the code for its definition, e.g., by putting it in a script file and running it as we saw before. Placing it in the shell startup script will make it available every time a shell session is started.

We can execute the alias or the function simply by its name `homedir`. In this case, the function would change the current working directory to our home directory and print its contents.

Of course, functions can do more than aliases. For instance, functions can take positional arguments that become available through special variables `$1`..`$2`..`$9` based on their position. Let's now modify our function to take a path to the directory instead:

```
anydir () {
    cd $1 ①
    ls
}
```

① We will expect the path to be the first positional argument of the function.

A function with positional arguments can be called the same way as any other program:

```
> anydir /folder/with/files
a.png  a.txt  b.png  b.txt ①
```

① This will be the output of the `ls` command. The current directory will be also changed.

Bash functions don't and can't return values like other programming languages. They can only end with an *exit status* using the `return` statement. Programs and functions should return 0 if all goes well and a non-zero number between 1 and 255 if not.

Let's modify the `anydir` function to return an error status code in case the directory doesn't exist:

```
anydir () {
    if [ ! -d $1 ]; then ①
      echo "$1 not a directory"
      return 1 ②
    fi
    cd $1
    ls ③
}
```

① We use `if` block to check whether the directory exists or not. `-d` means "check if the following directory exists". The `!` operator reverses the condition.

② We return the error exit status 1.

③ If the exit status is not provided by the `return` statement, the exit status of the last executed command will be used.

Execution of the anydir function

```
> anydir /not/existing/dir && echo "Directory exists"
/not/existing/dir not a directory
```

There is also the `exit` statement that can be used like `return`. It can be used outside of functions and will terminate the entire script where it is used, whereas `return` will only end the function, and a script that uses that function can continue to run.

Conditionals

Table 8. Conditional operators in Bash (< > characters indicate placeholders)

Operator	Checks
`<num> -eq <num2>`	Check if two numbers are equal
`<num> -lt <num2>`	Check if the first number is lower than the second
`<num> -gt <num2>`	Check if the first number is higher than the second
`<str> == <str2>`	Check if two strings are equal
`<str> != <str2>`	Check if two strings are different
`! <expression>`	Can be placed in front of an expression to reverse the condition
`-d <path>`	Check if the directory exists
`-e <path>`	Check if the file exists
`-r <path>`	Check if the file exists and we have permissions to read it
`-w <path>`	Check if the file exists and we have permissions to write to it
`-x <path>`	Check if the file exists and we have permissions to execute it

The table lists the most useful conditional operators used in conditional statements. They are used inside single or double brackets. Single brackets execute the inside expression literally, while double brackets allow for more logical comparisons and other features. Double brackets are usually recommended for preventing unexpected behavior.

Basic usage on the command line

```
> [[ 1 -eq 1 ]] && echo "This is equal"
This is equal
> [[ "str1" == "str2" ]] || echo "This is not equal"
This is not equal
```

Basic usage in a script

```
# template
if [[ EXPRESSION ]]; then
    # commands
fi

# example
if [[ 1 -eq 1 ]]; then
    echo "This is equals"
fi
```

Loops

Bash supports both `for` and `while` loops. In our final example, we will create a script that will accept a list of directories as positional arguments and list all files inside them with the `.sh` file extension. To do that, we are going to iterate over all arguments using the `for` loop.

All positional arguments passed to a script or a function are exposed together in a Bash array that we can access with `$@`. Working with arguments in a script file is the same as in functions.

listscripts.sh

```
#!/bin/bash

function list_scripts_in_dir() { ①
  ls $1 | grep ".*\.sh$"
}

for dir in "$@" ②
do ③
    echo "Scripts in dir $dir:"
    list_scripts_in_dir $dir ④
done
```

① At the beginning, we define a helper function that will list all `.sh` files inside a single folder. We offload the heavy lifting to `ls` and `grep` programs.

② We iterate over the array containing all positional arguments.

③ The `for` expression is followed by the `do`..`done` block that can contain commands as usual.

④ The `dir` variable now stores the current argument at each iteration. We can pass it as the positional argument to our helper function.

Once the script is saved in a file, we can run it:

```
> ./listscripts.sh ~/folder1 ~/folder2
Scripts in dir ~/folder1:
listscripts.sh
print.sh
Scripts in dir ~/folder2:
anotherscript.sh
```

Other Available Information

Besides accessing positional arguments with `$n` and `$@`, other information is exposed automatically to the script or function.

Placeholder	Meaning
`$0`	The name of the script
`$#`	The number of arguments
`$?`	The return code of the previous command
`$$`	The PID of the process running the script

Programming in Bash is as complex as programming in any other language. It is therefore impossible to squeeze it all in this book. However, the main concepts like scripts, functions, and some basic operations should now be clear.

The article Writing safe shell scripts[81] mentions some considerations we should take when writing shell scripts. We can also use a helpful online tool called Shellcheck[82] to automatically find bugs in shell scripts.

Writing Scripts in Other Languages

Many programming languages like Python, Ruby, or Perl can be used to write single-file executable scripts if they are installed on the system. This choice of another language is often desirable as many developers are not familiar with Bash syntax, and the code can become quite unreadable.

To use a different language in a script, we need to change the shebang line to point to the language executable. To make it more universal, we can leverage a program called `env` that will find the correct binary automatically, since the binary can be installed in different locations:

```
#!/usr/bin/env python
print("Robots are humans' best friends.")
```

Scheduling

The traditional Linux utility for scheduling jobs is cron[83]. It is a daemon process that runs continuously in the background and executes tasks scheduled using a file called *crontab*. It uses a special syntax for specifying execution times, but utilities like crontab guru[84] help us with that.

Cron is meant to be run on servers that are always powered on. So if a computer is not running at the right time, the scheduled job won't be executed. In this situation, we can use anacron[85], which is suitable for machines that are not always on.

A good use case for anacron is scheduling regular file backups on a personal computer. The *anacrontab* file in `/etc/anacrontab` reveals that some daily and weekly events are already configured to be run:

An example of anacrontab file

```
1   5   cron.daily      nice run-parts /etc/cron.daily ①
7   25  cron.weekly     nice run-parts /etc/cron.weekly
```

① The folder `/etc/cron.daily` contains scripts that will be executed daily. We only need to place our backup script there, and it will automatically run every day. Please note that this configuration is taken from a Fedora Workstation system, so that it might differ from yours.

[80] https://www.jefftk.com/icdiff

[81] https://sipb.mit.edu/doc/safe-shell/

[82] https://www.shellcheck.net/

[83] https://en.wikipedia.org/wiki/Cron

[84] https://crontab.guru/

[85] https://en.wikipedia.org/wiki/Anacron

Summary

1. Command-line interfaces are still relevant today. We can control computers by using shells such as Bash or Zsh inside terminal emulators. Terminal multiplexers and tiling emulators allow us to control multiple shell sessions simultaneously.
2. The command line is all about running commands. `PATH` environment variable controls what gets executed unless we specify relative or absolute paths to our scripts or binaries. Command-line arguments are used to set various options to programs that we want to run to control their behavior.
3. Standard input, standard output, and standard error streams allow programs to pass data from one to another, read from and write to files, or interact with the shell. The pipe `|` is the most useful operator when combining multiple programs.
4. It is good to know how to get help on the command line. Besides `--help` argument and accessing manual pages with `man`, another useful tool is `tldr` that can show us the most common usages for plenty of programs.
5. Using the command history, autocompletion, and shell shortcuts will allow us to find the right commands and their arguments faster. Using programs that can automatically jump to directories like `z` or correct our input will be efficient.
6. There are many specialized command-line utilities for all kinds of tasks from searching and viewing files, file management, job control, monitoring, package management, or scheduling.
7. Text data wrangling is an efficient way to alter the output of one program to use it in another or for data analysis and cleanup. Tools like `sed`, `awk`, or `jq` make many operations easy.
8. We can execute remote commands securely through SSH and transfer files with SCP when working with servers or other computers via a computer network.
9. Writing our Bash scripts is straightforward. The only thing needed is to specify which program should run the script and set the file as executable. After that, it is up to us to learn and use variables, conditionals, expansions, and other Bash language features.
10. We can configure our shell to our liking by setting defaults, themes, our aliases, and functions. We can do that using dotfiles. Tools like Oh My Zsh offer already tweaked shells.